Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

A Real-Time Dispatch and Tracking Platform for Volunteer Emergency Motorcycle Riders

By
Adam Gallagher

April 26, 2026

B.Sc. (Hons) in Software Development

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**

2 Methodology **5**
2.0.1 Development Approach 5
2.1 Sprint Planning and Task Management 6
2.2 Tools and Technologies 6
2.3 Individual Contribution 7
2.3.1 Testing Strategy 7
2.3.2 Simulation Approach 8
2.3.3 Justification of Methodology 8
2.3.4 Limitations of Methodology 9
2.4 Reflection on Development Process 9
2.5 Summary 10

3 Technology Review **11**
3.1 Volunteer Emergency Motorcycle Services 11
3.2 Digital Dispatch Systems 11
3.3 Web-Based Applications and PWAs 12
3.4 Backend Technologies 12
3.5 Cloud and Serverless Architectures 13
3.6 Data Storage Technologies 13
3.7 Real-Time Communication Technologies 13
3.8 Comparison of Design Approaches 14
3.9 Limitations of Existing Technologies 15
3.10 Summary 15

4 System Design **16**
4.1 System Overview 16
4.2 Architecture Overview 17
4.3 Frontend Design 18
4.4 Backend Design 19
4.5 API Design 20
4.6 Data Storage Design 21
4.7 Data Flow and Request Lifecycle 21
4.8 Real-Time Features 22
4.9 Implementation Details 23
4.10 System Reliability Considerations 24
4.11 Scalability Considerations 24
4.12 Design Trade-offs 25

5 System Evaluation **27**
5.1 Testing Overview 27
5.2 Unit Testing 27
5.3 Integration Testing 28
5.4 Simulation-Based Testing 28
5.5 Performance Evaluation 28
5.6 Reliability 29
5.7 Usability 29
5.8 Scalability Considerations 29
5.9 Comparison with Expected Outcomes 30
5.10 Limitations 30
5.11 Critical Discussion 31
5.12 Summary 31

6 Conclusion **33**

7 Appendix **35**
7.1 Source Code Repository 35
7.2 System Demonstration Video 35
7.3 Additional Notes 35

List of Figures

4.1 System architecture showing the interaction between frontend, API layer, backend, services, and data storage. 17

List of Tables

5.1 Evaluation of system objectives 30

Chapter 1

Introduction

Volunteer emergency motorcycle services, commonly referred to as Blood Bike organisations, provide an essential support function within healthcare logistics by transporting urgent medical items such as blood products, laboratory samples, and critical equipment between hospitals and clinics. These services are particularly important outside normal working hours, including nights, weekends, and public holidays, when standard transport options are often limited. Despite their importance, they are typically operated by volunteers who must work in demanding conditions with relatively limited technological support.

Motorcyclists are widely recognised as one of the most vulnerable groups of road users [1], facing a significantly higher risk of injury and fatality compared to occupants of enclosed vehicles. In the context of emergency volunteer riders, this risk is often compounded by the nature of the work. Riders frequently operate under time pressure, deal with unpredictable road and weather conditions, and rely on basic or improvised communication tools. While professional emergency services make extensive use of integrated digital systems for dispatch and coordination, similar levels of support are not usually available in volunteer-led organisations.

This project is motivated by that gap. It focuses on the design and development of a digital platform intended to support volunteer emergency motorcycle riders in their day-to-day operations. The system brings together several core functions, including job dispatch, real-time location tracking, rider availability management, and operational analytics, within a single web-based interface. The aim is not to replicate the complexity of large-scale emergency service systems, but rather to provide a lightweight and practical solution that can be realistically adopted in a volunteer setting.

From a technical perspective, the work spans several areas of computing, including distributed systems, web application development, and real-time data processing. The system is implemented as a full-stack application, with an Angular-based

Progressive Web Application (PWA) on the frontend and a Go-based backend deployed using a serverless cloud architecture. Persistent data is managed using a NoSQL database, while real-time features such as live tracking are handled through in-memory data structures and WebSocket communication. Design decisions throughout the project were guided by the need to balance performance, simplicity, and ease of deployment.

A particular focus of the system is its ability to support multiple users interacting concurrently, while maintaining a consistent and responsive user experience. Features such as real-time tracking and live updates require efficient handling of data and communication between components. To support this, the system makes use of asynchronous processing and lightweight communication mechanisms suited to real-time applications. In addition, a simulation tool was developed as part of the wider project to generate realistic usage scenarios and evaluate system behaviour under load.

In addition to improving operational efficiency, the introduction of a structured digital system has the potential to enhance transparency within volunteer organisations. By providing a centralised platform for managing jobs, tracking activity, and maintaining records, it becomes easier to monitor performance and identify areas for improvement. This is particularly important in environments where accountability and reliability are essential.

Furthermore, such systems support scalability. As demand for volunteer services increases, manual coordination becomes less effective. A digital platform enables organisations to manage increased workload without significantly increasing administrative effort.

The system was developed as part of a group project; however, this dissertation focuses on the aspects of the system that I contributed to, particularly in the areas of fleet management and human resource functionality.

The objectives of this dissertation are as follows:

- To examine the operational challenges and safety considerations associated with volunteer emergency motorcycle services.
- To design a system that supports coordination, tracking, and communication in this context.
- To implement a working platform using modern web and cloud technologies.
- To evaluate the system in terms of functionality, performance, and reliability.
- To reflect on the effectiveness of the solution and identify areas for further improvement.

The remainder of this dissertation is organised as follows. Chapter 2 describes the methodology adopted in this project. Chapter 3 reviews relevant technologies. Chapter 4 presents system design. Chapter 5 evaluates the system, and Chapter 6 concludes the work.

Chapter 2

Methodology

This chapter outlines the methodology used during the development of the system. It describes the development approach, the tools and processes used, and the methods applied to test and evaluate the system. As this was a group project, it also clarifies my individual contribution within the overall development process.

2.0.1 Development Approach

The project followed an Agile development methodology. This approach was chosen due to its flexibility and suitability for projects where requirements may evolve over time. In contrast to traditional methodologies such as Waterfall, Agile supports iterative development, enabling features to be implemented, tested, and refined in incremental cycles.

Agile methodologies are widely adopted in software development due to their ability to accommodate changing requirements and support continuous feedback [2]. This is particularly relevant in projects involving multiple interacting components, where early design decisions may need to be revisited as the system evolves.

The system was developed over a series of iterations, with each cycle focusing on a specific set of features. At the start of each iteration, tasks were identified and prioritised based on project requirements. These tasks were then implemented, tested, and reviewed before progressing to the next cycle.

This iterative approach allowed the team to respond effectively to challenges as they arose. For example, issues identified during testing could be addressed in subsequent iterations without requiring significant restructuring of the system. This was particularly beneficial in a project involving real-time features and distributed components, where changes in one area could impact others.

Agile was also well suited to the group-based nature of the project, as it enabled work to be divided into manageable tasks while maintaining overall coordination. Regular communication ensured that dependencies between components

were managed effectively.

In addition, the iterative process supported incremental validation of system functionality. Rather than deferring testing until the end of development, features were evaluated as they were implemented. This reduced the likelihood of major integration issues and contributed to overall system stability.

2.1 Sprint Planning and Task Management

Development was organised into informal sprint cycles, where tasks were grouped into short periods of focused work. While the project did not follow a strict Scrum framework, key Agile principles such as iterative development and continuous feedback were applied.

Tasks were identified based on system requirements and prioritised according to their importance. Core functionality, such as job management and authentication, was implemented first, followed by additional features such as fleet management and HR functionality.

Progress was tracked throughout development, with tasks being updated as they were completed. This helped to maintain visibility over the state of the project and ensured that important features were not overlooked.

Communication between team members played a key role in this process. Regular discussions allowed progress to be reviewed and potential issues to be identified early. This was particularly important given the interconnected nature of the system, where changes in one component could affect others.

One limitation of this approach was the lack of formal sprint documentation, such as sprint backlogs or review reports. While this did not significantly impact development, more structured tracking could have provided clearer insight into progress over time and supported more detailed evaluation of the development process.

2.2 Tools and Technologies

A range of tools and technologies were used to support development. The backend was implemented in Go, providing strong performance and support for concurrent processing. The frontend was developed using Angular, allowing for a structured and maintainable user interface.

Cloud services were used to support system functionality, including authentication, data storage, and email delivery. These services reduced the need for manual infrastructure management and allowed the system to scale as required.

Version control was used throughout the project to manage changes to the codebase. This allowed multiple team members to work on different parts of the system simultaneously while maintaining a consistent codebase. Changes could be reviewed and integrated in a controlled manner.

The selection of these tools was influenced by both technical and practical considerations. Technologies were chosen not only for their capabilities, but also for their suitability within a group project context. Ease of integration, availability of documentation, and familiarity within the team were all factors in this decision.

2.3 Individual Contribution

As this was a group project, responsibilities were divided across different areas of the system. My primary focus was on fleet management and HR-related functionality.

This included designing and implementing features for managing vehicle data, tracking availability, and maintaining service records. In addition, I contributed to the development of user management functionality, including handling user roles and supporting administrative processes.

While other team members worked on areas such as backend concurrency and tracking, it was necessary to ensure that all components integrated effectively. This required coordination between different parts of the system and an understanding of how changes in one area could affect others.

My contribution also involved ensuring that the user-facing aspects of these features were clear and usable. This required consideration of both technical implementation and user experience, ensuring that functionality could be accessed efficiently by administrative users.

2.3.1 Testing Strategy

Testing was an integral part of the development process and was conducted continuously rather than as a separate phase. This aligns with modern software engineering practices, where early and repeated testing is used to improve system reliability and reduce the cost of defect resolution.

Unit testing was used to verify individual components, ensuring that functions behaved as expected in isolation. Integration testing was used to validate interactions between components, confirming that data flowed correctly through the system.

In addition to these approaches, a simulation-based testing method was used to evaluate the system under conditions approximating real-world usage. This

allowed the system to be tested with multiple concurrent users, providing insight into its behaviour under load.

The use of multiple testing strategies reflects a layered approach to validation, where different aspects of the system are tested at different levels. This improves confidence in system correctness and reduces the likelihood of undetected defects.

Continuous testing also enabled early identification of issues, allowing them to be resolved before they affected other components. This contributed to the overall stability and robustness of the system.

2.3.2 Simulation Approach

A custom simulation tool was developed to generate concurrent user activity. This tool created multiple virtual users, each performing actions such as creating jobs, accepting tasks, and updating location data.

The simulation was designed to approximate real-world usage patterns by assigning different roles to users and varying their behaviour. This allowed the system to be evaluated under conditions that reflect realistic interaction scenarios.

Simulation-based testing is commonly used when real-world testing is not feasible, particularly in systems involving multiple users and time-dependent interactions. While simulation cannot fully replicate real-world conditions, it provides a controlled and repeatable environment for evaluating system performance and behaviour.

This approach allowed the identification of performance bottlenecks and potential issues in real-time functionality. It also enabled the system to be tested at a scale that would not have been possible using manual testing methods alone.

However, it is important to recognise that simulation has limitations. Factors such as network variability, user behaviour, and environmental conditions are difficult to replicate accurately. As a result, simulation should be viewed as a complementary testing method rather than a complete substitute for real-world evaluation.

2.3.3 Justification of Methodology

The choice of Agile methodology was appropriate for this project due to the evolving nature of the system requirements. As new features were introduced and existing features refined, the flexibility of an iterative approach allowed these changes to be incorporated without significant disruption.

Agile development supports continuous feedback and incremental improvement, which are particularly important in projects involving complex systems with multiple interacting components [2]. This made it well suited to the development of

a real-time system where design decisions needed to be evaluated and adjusted throughout the project lifecycle.

The use of simulation-based testing was also justified by the constraints of the project. Testing with real users was not feasible within the available time and resources, particularly given the specialised nature of the target user group. Simulation provided a practical alternative, allowing the system to be evaluated under controlled conditions.

While simulation does not fully replicate real-world usage, it enables repeatable testing and provides valuable insight into system behaviour under load. This makes it a suitable approach for evaluating performance and scalability within the scope of the project.

Overall, the chosen methodology provided a balance between flexibility, practicality, and technical validation, supporting the successful development and evaluation of the system.

2.3.4 Limitations of Methodology

While the chosen methodology was effective, it has several limitations.

The lack of formal Agile practices, such as structured sprint reviews and retrospective meetings, may have reduced opportunities for systematic evaluation of the development process. More formalised processes could have provided clearer insight into progress and supported continuous improvement.

In addition, simulation-based testing cannot fully replicate real-world usage. Factors such as network variability, user behaviour, and device performance may differ significantly in practice, potentially affecting system performance.

Another limitation is the absence of formal performance metrics. While system behaviour was observed during testing, precise measurements were not recorded. This limits the ability to quantify performance and compare results against established benchmarks.

Furthermore, the evaluation did not include feedback from real users. User involvement is an important aspect of system validation, particularly for applications intended for operational use. The absence of such feedback represents a limitation in assessing usability and practical effectiveness.

Despite these limitations, the methodology provided a suitable framework for developing and evaluating the system within the constraints of the project.

2.4 Reflection on Development Process

Reflecting on the development process, the use of an Agile methodology proved effective in managing the complexity of the project. The iterative nature of devel-

opment allowed features to be implemented and refined gradually, reducing risk and improving overall system quality.

However, the process could be improved through the introduction of more structured practices, such as formal sprint reviews and retrospective meetings. These would provide opportunities for systematic evaluation and continuous improvement.

The combination of Agile development and simulation-based testing provided a practical approach to system development. While not without limitations, it allowed the project to be completed successfully within the available time and resources.

2.5 Summary

This chapter has outlined the methodology used to develop the system, including the use of Agile practices, simulation-based testing, and collaborative development. These approaches allowed the system to be developed incrementally while maintaining flexibility and responsiveness to change.

Chapter 3

Technology Review

This chapter reviews the key technologies and concepts relevant to the development of the system. It examines existing approaches to similar problems and evaluates the tools and architectural decisions used in this project. The aim is to justify the selection of technologies and to highlight the trade-offs involved.

3.1 Volunteer Emergency Motorcycle Services

Volunteer emergency motorcycle services play a vital role in supporting healthcare systems, particularly in the transport of urgent medical items. These organisations operate with limited resources and rely heavily on volunteers, making efficiency and coordination essential.

Despite their importance, many such organisations continue to rely on manual coordination methods, such as phone calls or messaging applications. While these approaches may be sufficient for small-scale operations, they become increasingly inefficient as demand grows. The lack of structured systems can result in delays, miscommunication, and reduced visibility of operations.

This highlights the need for digital solutions that can support coordination and improve efficiency without introducing unnecessary complexity.

3.2 Digital Dispatch Systems

Digital dispatch systems are widely used in professional emergency services to coordinate resources and manage operations. These systems typically include features such as job assignment, tracking, and communication between users.

In recent years, there has been a shift towards web-based dispatch systems, which allow access from multiple devices without requiring specialised software.

Real-time tracking is often implemented using technologies such as WebSockets, enabling continuous updates between clients and servers.

However, many existing systems are designed for large-scale organisations and may be overly complex for volunteer-based services. This creates a gap for lightweight systems that provide essential functionality without the overhead of enterprise solutions.

3.3 Web-Based Applications and PWAs

Web-based applications have become increasingly popular due to their accessibility and ease of deployment. Progressive Web Applications (PWAs) extend traditional web applications by providing features such as offline support, push notifications, and improved performance.

Angular was selected as the frontend framework due to its structured approach and strong support for component-based development [3]. This makes it well suited for applications with multiple interacting features, as it allows components to be developed and maintained independently.

Compared to native mobile applications, PWAs offer several advantages. They can be accessed directly through a browser, eliminating the need for installation and simplifying updates. This is particularly important in a volunteer context, where users may have different devices and varying levels of technical expertise.

However, PWAs also have limitations. Performance may not match that of native applications in all scenarios, particularly when handling complex real-time interactions. Despite this, the benefits of accessibility and ease of deployment make PWAs a suitable choice for this project.

3.4 Backend Technologies

The backend of the system was implemented using Go, a programming language known for its performance and support for concurrency [4]. Go's lightweight concurrency model, based on goroutines, allows multiple tasks to be processed efficiently, making it well suited for applications that handle concurrent user interactions.

Alternative backend technologies, such as Node.js or Python, were also considered. While these offer flexibility and a wide range of libraries, Go provides advantages in terms of performance and simplicity. Its static typing and compiled nature also contribute to reliability and maintainability.

The choice of Go reflects the need for a backend that can handle multiple users and real-time updates without excessive complexity.

3.5 Cloud and Serverless Architectures

Cloud computing provides a flexible and scalable environment for deploying applications. In this project, a serverless architecture was used, allowing backend functionality to be executed without managing dedicated servers.

Serverless architectures offer several advantages. They automatically scale based on demand, reducing the need for manual resource management. They also simplify deployment, as developers can focus on application logic rather than infrastructure.

However, there are trade-offs. Serverless functions may experience cold start latency, which can affect performance when functions are invoked after periods of inactivity. In addition, developers have less control over the execution environment compared to traditional server-based architectures.

Despite these limitations, serverless architecture is well suited to this project due to its scalability and ease of deployment.

3.6 Data Storage Technologies

The system uses a NoSQL database (DynamoDB) for persistent data storage [5]. NoSQL databases provide flexibility in data structure, allowing the system to adapt to changing requirements.

Compared to relational databases, NoSQL systems offer better scalability and are well suited to distributed applications. However, they require careful design of data access patterns, as complex queries are not handled in the same way as in relational systems.

In addition to persistent storage, the system uses in-memory data structures for real-time features. This allows for faster data access and reduced latency, but introduces a risk of data loss if the system fails.

3.7 Real-Time Communication Technologies

Real-time communication is a key requirement of the system, particularly for features such as tracking and live updates. Technologies such as WebSockets enable continuous communication between the client and server, allowing updates to be delivered instantly.

This approach improves responsiveness and provides a better user experience. However, maintaining persistent connections for multiple users can increase system load, requiring efficient resource management.

Alternative approaches, such as polling, were considered but found to be less efficient due to the overhead of repeated requests. WebSockets provide a more

efficient solution for real-time communication.

3.8 Comparison of Design Approaches

Several design approaches were considered during the development of the system. These include web-based versus native applications, serverless versus traditional architectures, and NoSQL versus relational databases.

Web-based applications were chosen over native applications due to their accessibility and ease of deployment. While native applications may offer better performance, the benefits of cross-platform compatibility and reduced maintenance make web-based solutions more suitable for this project.

Serverless architecture was selected for its scalability and reduced infrastructure management. While it introduces some limitations, these are outweighed by the benefits in a project of this scale.

NoSQL databases were chosen for their flexibility and ability to handle distributed data. While they require careful design, they are well suited to the needs of the system.

These decisions reflect a balance between performance, scalability, and practicality, ensuring that the system meets its requirements without unnecessary complexity.

A more detailed comparison of these design approaches highlights the trade-offs that influenced the final system architecture. For example, while Node.js offers strong ecosystem support and ease of development, its single-threaded event loop model can introduce limitations when handling CPU-intensive tasks. In contrast, Go's concurrency model allows for more efficient parallel processing, making it better suited to applications involving real-time data updates and multiple simulataneous users.

Similarly, the choice between Progressive Web Applications and native mobile applications reflects a trade-off between performance and accessibility. Native applications typically provide superior performance and deeper integration with device hardware, which can be beneficial for features such as GPS tracking. However, they require platform-specific development and ongoing maintenance across multiple operating systems. PWAs, while slightly less performant in some scenarios, provide a single codebase that can be accessed across devices, reducing development overhead and simplifying deployment.

The decision to use a NoSQL database rather than a relational database also involved important considerations. Relational databases provide strong consistency and support complex queries, which can be advantageous in structured data environments. However, they are less flexible when dealing with rapidly evolving data models. NoSQL databases, such as DynamoDB, allow for more adaptable

data structures and are better suited to distributed systems, although they require careful design of access patterns to maintain efficiency.

These comparisons demonstrate that the chosen technologies were not necessarily optimal in all respects, but were selected based on their suitability for the specific requirements of the project. The final architecture represents a balance between performance, scalability, and practical constraints, rather than a focus on any single factor.

3.9 Limitations of Existing Technologies

While the technologies used provide strong capabilities, they are not without limitations. Web-based applications may struggle with performance in certain scenarios, particularly when handling complex real-time interactions.

Serverless architectures can introduce latency due to cold starts, and reliance on external services creates dependencies that may affect system reliability.

NoSQL databases require careful planning to ensure efficient data access, and real-time communication technologies can increase system load if not managed effectively.

These limitations highlight the importance of selecting technologies based on the specific requirements of the system and understanding the trade-offs involved.

3.10 Summary

This chapter has reviewed the technologies relevant to the system and discussed the trade-offs involved in their selection. The chosen technologies provide a suitable foundation for the system, balancing performance, scalability, and ease of deployment.

While there are limitations, the overall approach is appropriate for the requirements of the project and supports the development of a practical and effective solution.

Chapter 4

System Design

This chapter describes the design and implementation of the system. It outlines the overall architecture, explains how different components interact, and discusses the technical decisions made during development. The design was informed not only by the functional requirements of the project, but also by literature on volunteer emergency motorcycle safety, emergency response communication, real-time coordination, and safety-critical interface design.

The literature reviewed in Chapter 3 highlights that volunteer emergency motorcycle riders operate in a safety-sensitive environment where time pressure, environmental exposure, fatigue, and limited organisational infrastructure can increase operational risk [1, 6, 7]. As a result, the system was designed to provide lightweight digital support that improves coordination and situational awareness without introducing unnecessary complexity or cognitive burden.

4.1 System Overview

The system follows a full-stack architecture consisting of a client application, an API layer, a backend service, and supporting cloud-based infrastructure. These components work together to provide functionality such as job dispatch, real-time tracking, fleet management, and user administration.

At a high level, users interact with the system through a web-based interface implemented as an Angular Progressive Web Application (PWA). Requests are sent to the backend via an API Gateway, which handles authentication and routing. The backend processes these requests and interacts with data storage and external services.

This architecture was selected because volunteer emergency organisations require systems that are accessible, maintainable, and practical to deploy. Literature on emergency response information systems highlights the importance of

timely information exchange and situational awareness in supporting field operations [8, 9]. However, many professional emergency systems assume infrastructure and resources that are not always available in volunteer-led organisations. The system therefore adopts a lightweight architecture that provides essential coordination features without replicating the complexity of enterprise dispatch systems.

Real-time features are supported through continuous communication between the client and backend, allowing updates to be reflected immediately in the user interface. This supports dispatcher awareness of rider availability and job status, which is important in time-critical operational settings. Rather than focusing only on technical performance, the design prioritises operational clarity, reduced communication overhead, and ease of adoption.

In designing the system, particular emphasis was placed on maintaining a clear separation between components. This ensures that each part of the system has a well-defined responsibility, improving maintainability and reducing complexity. By structuring the system in this way, it becomes easier to extend or modify individual components without affecting the entire architecture.

4.2 Architecture Overview

The architecture of the system is shown in Figure 4.1. It illustrates the interaction between the client, API layer, backend, services, and data storage.

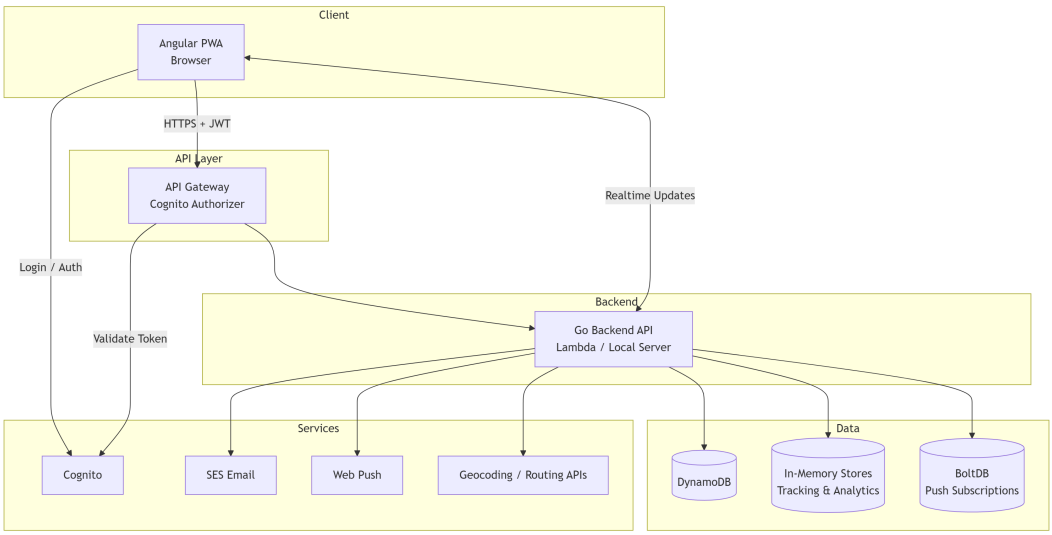

Figure 4.1: System architecture showing the interaction between frontend, API layer, backend, services, and data storage.

The client communicates with the backend using HTTPS requests that include

a JSON Web Token (JWT) for authentication. The API Gateway validates this token using Cognito before forwarding the request to the backend service.

The backend processes the request and interacts with various components, including databases, in-memory storage, and external services. Real-time updates are sent back to the client to ensure that the interface reflects the current state of the system.

This layered approach provides several advantages. It isolates concerns such as authentication, routing, and business logic, making the system easier to understand and maintain. It also allows for scalability, as individual layers can be optimised or replaced independently if required.

A key aspect of this architecture is the separation between authentication and application logic. By delegating authentication to the API Gateway and Cognito, the backend service is not required to manage user identity directly. This reduces complexity within the application code and ensures that security concerns are handled using a managed and well-tested service.

The use of JWTs enables stateless authentication, meaning that each request contains all the information required to verify the user. This eliminates the need for server-side session storage and allows the system to scale more effectively. JWT-based authentication is also suitable for distributed systems because it reduces server-side state management while supporting secure access control [10]. However, this approach also requires careful validation of tokens to ensure that expired or invalid tokens are not accepted.

Another important consideration is the flow of requests through the system. When a request is made, it passes through several layers before reaching the backend logic. While this introduces a small amount of overhead, it provides benefits in terms of security, scalability, and maintainability. Each layer performs a specific role, ensuring that responsibilities are clearly separated.

This design also supports flexibility in deployment. For example, the backend can be run locally during development or deployed as a serverless function in a production environment. This dual-mode operation simplifies testing and allows the system to be adapted to different environments without significant changes.

4.3 Frontend Design

The frontend was implemented as an Angular PWA. This approach allows the system to be accessed through a browser while still providing features such as responsiveness and push notifications.

The decision to use a PWA was informed by the operational context of volunteer emergency riders. Literature on Progressive Web Applications identifies their suitability for cross-platform access and simplified deployment, particularly

where users may rely on different devices [11]. This is relevant in a volunteer setting, where riders and administrators may use personal mobile phones, tablets, or desktop devices rather than standardised organisational hardware.

The application follows a component-based architecture, where each feature is encapsulated within its own component. This improves maintainability and allows different parts of the system to be developed independently.

Routing is used to manage navigation between views, including dashboards, job management, fleet management, and administrative interfaces. Access to these routes is controlled based on user roles, ensuring that users can only access relevant functionality.

State management is handled within services, which act as intermediaries between components and the backend API. This allows data to be shared across components while maintaining a clear separation of concerns.

From a design perspective, particular attention was given to usability. The interface was structured to ensure that key actions, such as accepting jobs or updating status, could be performed quickly and efficiently. Human-computer interaction research in emergency medical information systems highlights the importance of clear interface design, low cognitive load, and rapid comprehension in time-critical environments [12]. These principles influenced the decision to keep workflows direct and role-specific, reducing the amount of information presented to users at any one time.

This is particularly important for riders, who may interact with the system before or after journeys, or in operational contexts where excessive interface complexity could increase distraction. The frontend therefore prioritises concise information, predictable navigation, and minimal unnecessary interaction.

4.4 Backend Design

The backend is implemented as a RESTful API written in Go. It is responsible for handling incoming requests, applying business logic, and interacting with data storage and external services.

The backend follows a modular architecture, with separate components for routing, authentication, and domain-specific functionality. This improves maintainability and allows individual components to be modified independently.

Concurrency is handled using Go's goroutines, enabling the backend to process multiple requests simultaneously. This is particularly important for real-time features, where updates from multiple users must be handled efficiently.

Middleware is used to handle cross-cutting concerns such as authentication and logging. This ensures consistent behaviour across all endpoints.

The choice of Go was influenced by its performance characteristics and simplicity [4]. Compared to other backend technologies, Go provides efficient concurrency handling with relatively low overhead. This makes it well suited to applications that require real-time updates and concurrent user interaction.

In addition to its concurrency model, Go was selected due to its simplicity and strong standard library support. This reduces reliance on external dependencies, which can introduce additional complexity and potential security risks. The language's emphasis on simplicity also makes the codebase easier to understand and maintain, particularly in a group development environment.

The backend was designed to handle both synchronous and asynchronous operations. Standard API requests follow a request-response pattern, while real-time updates are processed asynchronously. This separation ensures that time-sensitive operations, such as tracking updates, do not block other parts of the system.

Error handling was also considered during backend design. Rather than allowing failures to propagate, errors are captured and returned as structured responses. This improves system robustness and provides clearer feedback to the frontend, allowing appropriate handling of different error conditions.

Logging was incorporated to support debugging and monitoring. By recording key events and errors, it becomes easier to identify issues during development and testing. While this logging is relatively simple, it provides a foundation for more advanced monitoring in future iterations of the system.

4.5 API Design

The system exposes a set of RESTful endpoints that allow the frontend to interact with backend functionality. Each endpoint corresponds to a specific resource or action.

- `GET /jobs` retrieves job data
- `POST /jobs` creates new jobs
- `PUT /jobs/{id}` updates existing jobs
- `DELETE /jobs/{id}` removes jobs

Requests and responses are formatted using JSON, providing a consistent structure for communication. Error handling is implemented using standard HTTP status codes.

The API design prioritises simplicity and clarity. Endpoints are structured in a predictable manner, making them easier to use and understand. This is

particularly important when integrating frontend components, as it reduces the likelihood of errors and simplifies development.

REST was selected because it provides a simple and widely supported communication model for client-server applications. Fielding's REST architectural model emphasises stateless communication and uniform interfaces, both of which support scalability and maintainability in distributed systems [13]. While alternatives such as GraphQL or event-driven architectures could provide more flexible data retrieval or asynchronous communication, they would also introduce additional complexity. Given the scale and scope of this project, REST provided an appropriate balance between capability, simplicity, and ease of implementation.

4.6 Data Storage Design

The system uses a combination of persistent and in-memory storage. Persistent data is stored in DynamoDB, including users, jobs, and fleet records [5].

Real-time data, such as rider location, is stored in memory. This allows for faster updates and reduced latency but introduces a risk of data loss.

The separation of data types improves performance while ensuring that critical data is stored reliably.

This hybrid approach reflects a trade-off between performance and reliability. While in-memory storage provides significant performance benefits, it requires careful management to ensure that data remains consistent and recoverable where necessary.

This decision was influenced by the distinction between critical operational records and transient coordination data. Job records, user information, and fleet data require persistence because they support accountability and administrative oversight. In contrast, live tracking information is primarily used to support short-term situational awareness. Literature on emergency response systems highlights the importance of timely and accessible information during field operations [8, 9]. For this reason, reducing latency for live updates was prioritised, while persistent storage was reserved for data that must be retained over time.

4.7 Data Flow and Request Lifecycle

Understanding how data flows through the system is essential for evaluating its design.

When a user performs an action, such as creating a job, the frontend sends a request to the API Gateway. The request is authenticated and forwarded to the

backend, where it is processed and stored in the database. A response is then returned to the client.

For real-time features, data is processed in memory and sent directly to connected clients. This reduces latency and improves responsiveness.

This flow ensures that the system can handle both standard request-response interactions and real-time updates efficiently.

The request lifecycle is particularly important when considering system performance. Each step in the process introduces latency, and the overall response time depends on the efficiency of each component. By minimising unnecessary processing and ensuring that each layer performs only its intended function, the system is able to maintain responsive performance.

In addition, the separation between real-time and persistent data flows reduces the load on the database. Rather than writing every update to persistent storage, temporary data is handled in memory and only essential information is stored. This approach improves performance but requires careful management to ensure data consistency.

Another consideration is fault handling within the request lifecycle. If a failure occurs at any stage, the system must respond appropriately. For example, if authentication fails, the request should be rejected immediately. If a database operation fails, the system should return an error without affecting other operations. This ensures that failures are contained and do not impact the overall stability of the system.

The request lifecycle also reflects the need to reduce cognitive and operational friction for users. In emergency and time-sensitive environments, delays or unclear feedback can negatively affect decision-making. Research on emergency decision-making highlights the importance of timely observation, decision, and action cycles under uncertainty [14]. The system therefore aims to keep user interactions short, provide clear state updates, and ensure that operational changes are visible quickly to relevant users.

4.8 Real-Time Features

Real-time functionality is implemented through continuous communication between the client and backend. Updates are processed in memory and distributed to users.

This approach ensures that changes are reflected immediately in the interface, improving usability and responsiveness.

Real-time visibility was prioritised because dispatcher awareness of rider status and job progress is central to effective coordination. Literature on first responder communication systems highlights the importance of resilient and timely informa-

tion exchange in dynamic field environments [8]. Similarly, IoT-based emergency medical information systems demonstrate how improved data availability can support response coordination [9]. Although this project does not attempt to reproduce a full professional emergency service platform, it applies the same principle of improving coordination through timely operational visibility.

However, real-time communication requires careful management to avoid performance issues, particularly when handling multiple concurrent users. Efficient handling of connections and data updates is essential to prevent bottlenecks.

Real-time communication introduces additional complexity compared to traditional request-response systems. Maintaining persistent connections requires efficient resource management, particularly when the number of concurrent users increases. Each connection consumes system resources, and poorly managed connections can lead to performance degradation.

To address this, the system uses lightweight communication mechanisms and avoids unnecessary data transmission. Only relevant updates are sent to clients, reducing network overhead and improving efficiency. This selective update approach ensures that the system remains responsive even as activity increases.

Another challenge is ensuring consistency between clients. Since updates are delivered in real time, there is a risk that different users may briefly see different states of the system. While this is generally acceptable for non-critical data, it highlights the importance of designing real-time systems with eventual consistency in mind.

The design therefore reflects a trade-off between responsiveness and persistence. For live location and availability data, immediate visibility is more important than long-term retention. This is consistent with the system's purpose: to support coordination and situational awareness during active operations, rather than to provide a permanent audit trail of every transient update.

4.9 Implementation Details

The backend is structured into layers, including the API layer, service layer, and data access layer. This separation improves maintainability and allows components to be tested independently.

Authentication is implemented using JWTs, which are validated for each request. User roles are extracted from the token and used to control access to system features.

Data models are designed to support efficient access, with records stored in DynamoDB using key-value structures. Real-time data is stored in memory and updated frequently.

External services are integrated through the backend, including authentication providers, email services, and routing APIs.

This layered implementation ensures that the system remains organised and scalable, allowing new features to be added with minimal disruption.

4.10 System Reliability Considerations

Reliability was a key consideration during design. The system includes error handling mechanisms to ensure that failures do not cause crashes.

Cloud services provide built-in redundancy, improving reliability. However, dependencies on external services introduce potential risks.

To address this, the system is designed to degrade gracefully. Non-critical features can fail without affecting core functionality.

Reliability is particularly important because volunteer emergency motorcycle riders may operate in variable network and environmental conditions. The literature highlights that emergency response communication systems must remain usable under dynamic and resource-constrained conditions [8]. While this project does not provide complete offline operation, the architecture separates critical persistent records from transient real-time data so that failure in one area does not necessarily compromise the entire system.

4.11 Scalability Considerations

The system is designed to scale through the use of serverless infrastructure. Resources can be allocated dynamically based on demand, allowing the system to handle increased load.

However, scaling introduces challenges, particularly for real-time features. Managing large volumes of concurrent updates requires efficient resource usage.

Scalability is influenced not only by infrastructure but also by system design. Efficient handling of data and minimisation of unnecessary processing are essential for maintaining performance as the system grows. For example, reducing database queries and using caching mechanisms can significantly improve scalability.

The use of serverless infrastructure simplifies scaling at the infrastructure level, but application-level considerations remain important. Poorly designed logic can still create bottlenecks, regardless of available resources. As a result, scalability must be considered throughout the design process, rather than relying solely on infrastructure to address performance issues.

Future improvements could include the introduction of distributed data processing or message queues to handle high volumes of real-time updates. These

approaches would allow the system to scale more effectively in larger deployments.

Despite these challenges, the architecture provides a strong foundation for scaling.

4.12 Design Trade-offs

Several trade-offs were considered:

- Simplicity vs functionality
- Performance vs persistence
- Flexibility vs complexity

These trade-offs reflect the constraints of the project and the need to balance different requirements.

The most important trade-off was between implementing a feature-rich system and maintaining a lightweight tool suitable for volunteer use. Literature on safety-critical interface design emphasises that systems used in time-sensitive environments should reduce cognitive load rather than introduce additional complexity [12]. For this reason, the system prioritises core workflows such as job dispatch, tracking, fleet management, and user administration rather than attempting to replicate all features of professional emergency service platforms.

A second trade-off involved performance versus persistence. Live tracking data was stored in memory to reduce latency, while critical operational records were stored persistently in DynamoDB. This allows the system to support real-time situational awareness while preserving essential administrative data.

A third trade-off involved scalability versus development complexity. Serverless cloud infrastructure reduces the need for manual server management and supports dynamic scaling [15]. However, it also introduces dependencies on external services and potential latency issues. Within the constraints of the project, this was considered acceptable because the system targets volunteer organisations that require maintainable and cost-effective deployment rather than highly customised infrastructure.

Overall, these design trade-offs demonstrate that the final architecture was not selected because it was optimal in every technical dimension. Instead, it was selected because it best matched the operational, usability, and resource constraints identified in the literature and through the project requirements.

4.13 Summary

This chapter has described the design and implementation of the system. The architecture combines a responsive frontend, a modular backend, and scalable infrastructure to provide a practical solution for volunteer emergency motorcycle services.

By linking the design decisions to literature on motorcycle safety, emergency communication, human-computer interaction, and safety-critical systems, this chapter demonstrates that the architecture was shaped by the specific constraints of volunteer emergency operations. The resulting system provides lightweight digital support for coordination, visibility, and administration while avoiding unnecessary complexity.

Chapter 5

System Evaluation

This chapter evaluates the system in terms of testing, performance, reliability, usability, and scalability. The evaluation assesses both the technical correctness of the system and its effectiveness in supporting coordination within a safety-critical operational environment.

The evaluation is informed by literature on emergency response systems and human–computer interaction, which emphasises the importance of situational awareness, low-latency communication, and reduced cognitive load in time-sensitive environments [8, 12].

5.1 Testing Overview

A structured testing approach was adopted to ensure system correctness and reliability. This included unit testing, integration testing, and simulation-based testing.

Each level of testing was designed to validate a different aspect of the system, from individual components to full system workflows. This layered approach ensures that faults can be identified early and that system behaviour remains consistent as components are integrated.

5.2 Unit Testing

Unit testing was used to verify the correctness of individual components in isolation. Backend functions, API endpoints, and data handling logic were tested independently to ensure expected behaviour.

This approach helped identify logical errors early in development and ensured that each component functioned correctly before integration. Unit testing improves reliability by reducing the likelihood of defects propagating through the system.

5.3 Integration Testing

Integration testing was conducted to ensure that system components interacted correctly. This included testing communication between the frontend, API Gateway, backend services, and database.

These tests verified that requests were correctly processed, authenticated, and responded to, and that data consistency was maintained across the system. Integration testing was particularly important for validating authentication workflows and API communication.

5.4 Simulation-Based Testing

Simulation-based testing was used to evaluate system behaviour under conditions approximating real-world usage. Multiple users were simulated interacting with the system concurrently, allowing observation of system responsiveness and stability.

This approach is particularly useful in the absence of real-world deployment, as it allows testing of coordination features and system performance under load. It also enables identification of potential bottlenecks in real-time functionality.

5.5 Performance Evaluation

System performance was evaluated during simulation testing with multiple concurrent users.

The system demonstrated responsive behaviour, with minimal delay observed in operations such as job updates and tracking. The use of in-memory data storage for real-time features reduced latency by avoiding frequent database access.

This design aligns with research on emergency response systems, which highlights the importance of timely information delivery for effective coordination [8]. In time-critical environments, delays in information flow can negatively affect decision-making.

Serverless infrastructure also contributed to performance by enabling dynamic scaling of resources based on demand.

However, precise performance metrics were not recorded. While observational results indicate effective performance, the lack of quantitative data limits formal benchmarking. Future work could include measuring response times and system throughput.

5.6 Reliability

System reliability was assessed through extended testing and simulation. The system remained stable during continuous operation, with no critical failures observed.

Error handling mechanisms ensured that invalid inputs were managed appropriately and did not cause system crashes. Instead, structured error responses were returned to the user.

The use of cloud-based services improves reliability through redundancy and fault tolerance. However, reliance on external services introduces dependencies that may impact availability.

Real-time data stored in memory introduces a risk of data loss. This reflects a trade-off between performance and persistence. Literature suggests that in emergency systems, immediate access to operational data is often more important than long-term storage [9].

5.7 Usability

Usability was evaluated through interaction with the system during development and testing.

The interface was designed to be clear and task-focused, allowing users to perform actions such as job assignment and status updates with minimal interaction. Navigation is structured to reduce complexity and improve efficiency.

This approach aligns with human–computer interaction research, which emphasises the importance of low cognitive load in safety-critical systems [12].

The use of a web-based interface ensures accessibility across multiple devices, which is important in a volunteer context where users rely on personal hardware.

However, formal usability testing was not conducted. Future work could include structured user testing to provide more objective usability evaluation.

5.8 Scalability Considerations

The system was tested with approximately 90 concurrent users. Under this workload, it maintained stable performance and responsiveness.

The serverless infrastructure allows the system to scale dynamically, making it suitable for environments with variable usage patterns.

However, scaling real-time features presents challenges. As user numbers increase, managing concurrent updates efficiently becomes more complex.

Future improvements could include distributed processing or message queuing to support higher scalability.

5.9 Comparison with Expected Outcomes

The system was designed to meet several key objectives, including improving coordination, enabling real-time tracking, and providing an accessible platform for volunteer emergency services.

Objective	**Status**	**Justification**
Real-time tracking	Achieved	Implemented and tested
Coordination system	Achieved	Functional across all users
Scalability	Partial	Tested at moderate scale
Performance metrics	Partial	No quantitative data collected

Table 5.1: Evaluation of system objectives

Overall, the system demonstrates a strong level of alignment with its intended objectives. Core functionalities such as real-time tracking and coordination were fully achieved, enabling effective communication and operational visibility between users. These features directly address challenges identified in the literature, particularly the need for improved situational awareness and timely information flow in emergency response environments [8].

However, some objectives were only partially achieved. While the system demonstrates good performance under simulated conditions, the absence of quantitative performance metrics limits the ability to formally validate efficiency. Similarly, scalability was demonstrated at a moderate level but has not been tested under large-scale deployment scenarios. These limitations highlight areas for future improvement and suggest that further evaluation would be required to fully validate the system in real-world contexts.

Overall, the system provides a practical and effective solution within the scope of the project, balancing performance, usability, and accessibility while addressing the constraints of volunteer emergency operations.

Although performance and usability goals were met on the tested scale, more evaluation would be required to confirm effectiveness in real-world deployment.

5.10 Limitations

Several limitations were identified during evaluation.

First, testing was conducted in a simulated environment, which does not fully capture real-world conditions such as network variability and user behaviour.

Second, quantitative performance metrics were not collected, limiting the ability to perform detailed analysis.

Third, the system has not been deployed in an operational environment, meaning that its effectiveness in real-world scenarios remains unverified.

Finally, usability evaluation was informal and based on developer interaction rather than structured user testing.

5.11 Critical Discussion

The evaluation demonstrates that the system meets its functional requirements and performs effectively under simulated conditions. However, several trade-offs and limitations must be considered.

The system prioritises real-time responsiveness over data persistence, reflecting the importance of situational awareness in emergency contexts. While this improves coordination, it introduces potential risks related to data loss.

The use of lightweight, web-based technologies makes the system accessible and easy to deploy, but may limit functionality compared to enterprise-level emergency systems.

An additional limitation relates to the absence of real-world deployment. While simulation-based testing provides useful insights into system behaviour, it cannot fully replicate the complexity and variability of real operational environments. Factors such as network instability, device variability, and human decision-making under pressure may significantly influence system performance in practice.

Furthermore, the evaluation did not include direct feedback from end users, such as volunteer riders or coordinators. Incorporating user feedback would provide valuable insight into usability, practicality, and acceptance of the system, particularly in a volunteer context where ease of use is critical.

These limitations suggest that while the system performs effectively within the scope of this project, further validation is required to confirm its suitability for real-world deployment.

Despite these limitations, the system provides a practical solution tailored to volunteer emergency motorcycle services. By aligning design decisions with principles identified in the literature, the system addresses key challenges related to coordination, usability, and operational awareness.

5.12 Summary

This chapter evaluated the system across multiple dimensions, including testing, performance, reliability, usability, and scalability.

The results indicate that the system performs effectively within the scope of this project and successfully supports key operational requirements. While limitations

exist, particularly regarding real-world validation and quantitative measurement, the system provides a strong foundation for further development and deployment.

Chapter 6

Conclusion

This dissertation has explored the design, implementation, and evaluation of a system intended to support volunteer emergency motorcycle services. These organisations provide an essential role in healthcare logistics, yet often operate without the technological support available to professional emergency services. This project aimed to address that gap by developing a practical and accessible digital platform.

The system integrates several key features, including job dispatch, real-time tracking, fleet management, and user administration, within a single web-based application. By combining these functions, the system provides a more structured approach to coordination, reducing reliance on informal communication methods.

From a technical perspective, the project demonstrates the application of modern web and cloud technologies to a real-world problem. The use of a Progressive Web Application ensures accessibility across devices, while the Go-based backend provides efficient handling of concurrent requests. The adoption of a serverless architecture allows the system to scale dynamically, reducing the need for manual infrastructure management.

The evaluation of the system indicates that it performs effectively under simulated conditions. Testing demonstrated that the system is capable of handling concurrent users while maintaining responsiveness. Real-time features functioned as intended, and the system remained stable during extended testing.

However, several limitations should be acknowledged. The system has not been deployed in a real operational environment, meaning that performance under real-world conditions has not been fully validated. Additionally, the use of in-memory data for real-time tracking introduces a risk of data loss, and reliance on external services creates dependencies that may affect reliability.

Despite these limitations, the system provides a strong foundation for further development. The modular architecture allows for future enhancements without significant restructuring, and the use of widely adopted technologies ensures that

the system can be maintained and extended over time.

Future work could involve deploying the system within a volunteer organisation to gather real-world feedback. This would provide valuable insight into usability and performance, allowing further refinement of the system. Additional features, such as advanced analytics or improved mobile integration, could also be explored.

Another potential area for improvement is the persistence of real-time data. While in-memory storage provides performance benefits, alternative approaches could be considered to improve reliability without significantly increasing latency. This could involve hybrid storage solutions or more advanced data synchronisation techniques.

In reflecting on the project as a whole, it is clear that the development of such systems requires careful consideration of both technical and practical factors. Balancing performance, usability, and scalability is essential to ensure that the system meets the needs of its users.

In addition to these areas, the broader impact of systems such as this should also be considered. By introducing structured digital tools into volunteer-based organisations, there is potential to improve not only operational efficiency but also overall safety and coordination. Enhanced visibility of rider activity and job status can reduce uncertainty and support better decision-making during time- critical operations.

Furthermore, the use of modern, accessible technologies may encourage wider adoption of digital systems within similar organisations. This could contribute to more consistent standards of operation and improved collaboration between different services. While the system developed in this project is relatively small in scale, it demonstrates how targeted technological solutions can provide tangible benefits in real-world contexts.

Overall, this project demonstrates that it is possible to develop a system that supports the coordination and efficiency of volunteer emergency motorcycle services using modern technologies. While further work is required to fully realise its potential, the system represents a meaningful step towards improving the effectiveness of these important services.

Chapter 7

Appendix

7.1 Source Code Repository

The full source code for the system is available in the following GitHub repository:

- **GitHub Repository:** `https://github.com/AdamGallagher339/Codename-Blood`

The repository includes both frontend and backend implementations, along with configuration files and documentation required to run the system.

7.2 System Demonstration Video

A screencast demonstrating the functionality of the system has been provided as part of this submission.

- **Video Link:** `https://youtu.be/NlmAewMhAWk`

The video demonstrates key features of the system, including:

- Job creation and assignment
- Real-time tracking and updates
- Fleet management functionality
- User and role management

7.3 Additional Notes

The system may require configuration of environment variables and cloud services to run locally or in a deployed environment. Instructions for setup and deployment are included within the repository.

Bibliography

[1] World Health Organization. Global status report on road safety 2018. https://iris.who.int/handle/10665/276462, 2018.

[2] Kent Beck et al. Manifesto for agile software development, 2001.

[3] Google. Angular documentation. https://angular.io/docs, 2024.

[4] Google. The go programming language. https://go.dev/doc/, 2024.

[5] Amazon Web Services. Amazon dynamodb documentation. https://docs.aws.amazon.com/dynamodb/, 2024.

[6] George Kiwango et al. A systematic review of risk factors associated with road traffic crashes and injuries among commercial motorcycle drivers. *International Journal of Injury Control and Safety Promotion*, 31(4):332–345, 2024.

[7] P. Wisutwattanasak et al. Factors affecting single and multivehicle motorcycle crashes. *Big Data and Cognitive Computing*, 8(10):128, 2024.

[8] Athanasios Douklias, Maria Krommyda, and Angelos J. Amditis. Resilient communications for the first responders at the field. In *2021 Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS)*, pages 37–42, 2021.

[9] Boyi Xu et al. Ubiquitous data accessing method in iot-based information system for emergency medical services. *IEEE Transactions on Industrial Informatics*, 10(2):1578–1586, 2014.

[10] Michael Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, 2015.

[11] Andreas Biørn-Hansen, Tim A. Majchrzak, and Tor-Morten Grønli. Progressive web apps: The future of web development? *Proceedings of the International Conference on Web Engineering*, 2017.

[12] Y. Batu Salman, Hong-In Cheng, and Patrick E. Patterson. Icon and user interface design for emergency medical information systems: A case study. *International Journal of Medical Informatics*, 81(1):29–35, 2012.

[13] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[14] Kaifeng Feng et al. Analysis of emergency decision-making patterns in civil aviation risk events based on the observe–decide–act model. *Reliability Engineering & System Safety*, 270:112177, 2025.

[15] Amazon Web Services. Serverless architectures on aws. `https://aws.amazon.com/serverless/`, 2023.